

Smart TecnoHouse SA

Memoria Técnica – Programación Orientada a Objetos

Desiderio Hitos Padial | UTAMED | 2025-2026 | Repositorio: <https://github.com/hitosdesi/SmartTecnoHouse-POO>

SECCIÓN 1 – Decisiones de Diseño y Principios SOLID

El sistema se estructura sobre una jerarquía de interfaces y clases abstractas que explota el polimorfismo. La clase raíz `IDispositivo` define el contrato mínimo (`getID`, `getNombre`, `getEstadoActual`) del que heredan `Sensor` y `Actuador`, y de éstas las implementaciones concretas.

Principio	Aplicación	Clase
S – Responsabilidad Única	Cada clase tiene una sola razón de cambio	SmartTecnoHouse, GestorLog, GestorPersistencia
O – Abierto/Cerrado	Nuevos dispositivos sin modificar código existente	SensorHumedad, ActuadorEnchufe
L – Sustitución Liskov	Todo Sensor/Actuador puede tratarse como IDispositivo	Lis SmartTecnoHouse
I – Segregación Interfaces	IDispositivo solo define métodos mínimos comunes	interface IDispositivo
D – Inversión Dependencias	Controlador depende de abstracciones, no implementaciones	Regla, IDispositivo

SECCIÓN 2 – Patrones de Diseño

2.1 Patrón Strategy

Se define la interfaz `Regla` con el método `void aplicar(List<Sensor>, List<Actuador>)`. Tres implementaciones encapsulan lógica independiente: `ReglaVentilacion` (`temp > 26°C → fan HIGH`), `ReglaIluminacion` (`luz < 200 lux + presencia → bulb ON`), `ReglaAhorro` (`sin presencia → apagar todo`). `SmartTecnoHouse` itera una `List<Regla>` y llama `aplicar()` en cada una sin conocer la implementación concreta.

2.2 Patrón MVC

Capa	Paquete	Responsabilidad
Modelo	modelo/	Lógica de dominio pura. Sin ningún import javax.swing.*
Vista	vista/	Clases Swing (JFrame, JPanel). Solo pinta datos y captura eventos
Controlador	controlador/	Media entre Modelo y Vista. Implementa ActionListeners

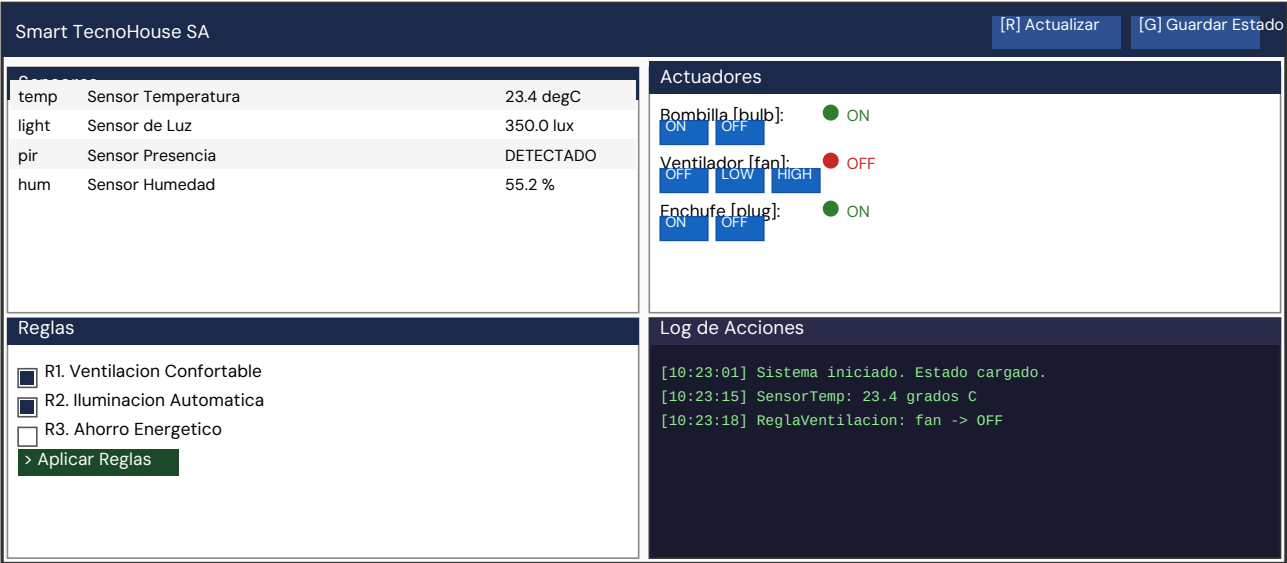
2.3 Patrón Singleton

`SmartTecnoHouse` y `GestorLog` implementan Singleton (constructor privado + `getInstance()` sincronizado) para garantizar una única instancia compartida entre Controlador y Vista.

SECCIÓN 3 — Manual de Usuario

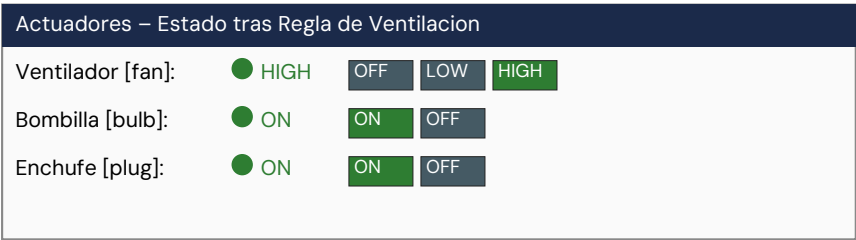
La interfaz gráfica se compone de cuatro zonas funcionales. A continuación se describe cada una junto con su representación visual.

Captura 1 — Ventana Principal



Zona	Función
Panel Sensores	Tabla actualizable con ID, nombre y valor en tiempo real de los 4 sensores
Panel Actuadores	Botones ON/OFF/HIGH por actuador. Estado en verde (ON) o rojo (OFF)
Panel Reglas	Checkboxes para activar reglas. Botón "Aplicar Reglas" ejecuta automatización
Log de Acciones	Registro en tiempo real de todas las acciones con marca de tiempo

Captura 2 — Detalle Panel Actuadores (tras Regla Ventilación)



Tras pulsar '[R] Actualizar' con temperatura 28.1°C, la ReglaVentilacion activó el ventilador a HIGH automáticamente. El estado se muestra en verde para ON/HIGH y rojo para OFF.

SECCIÓN 4 — Flujo de Uso

1. Iniciar aplicación → se carga data/estado.json automáticamente
2. Panel Sensores: pulsar '[R] Actualizar' para obtener nuevas lecturas simuladas
3. Panel Actuadores: hacer clic en ON/OFF/HIGH para control manual directo
4. Panel Reglas: marcar reglas deseadas y pulsar '> Aplicar Reglas' para automatización
5. Botón '[G] Guardar Estado' → persiste el estado en data/estado.json
6. Al cerrar la ventana → el estado se guarda automáticamente

SECCIÓN 5 — Ampliaciones: Nuevos Dispositivos

Para demostrar el Principio Abierto/Cerrado, se han añadido un nuevo sensor y un nuevo actuador sin modificar ninguna clase existente del sistema.

SensorHumedad (NUEVO)	ActuadorEnchufe (NUEVO)
ID: "hum" Clase: SensorHumedad extends Sensor Rango: 20-90 % humedad relativa Método: actualizarValor() genera valor aleatorio getEstadoActual(): "55.2 %" Justificación: Añadido sin tocar Sensor, SensorTemperatura, ni SmartTecnoHouse. Solo se instancia en el constructor del Singleton.	ID: "plug" Clase: ActuadorEnchufe extends Actuador Acciones: ["ON", "OFF"] getEstadoActual(): "ON" o "OFF" Integrado en ReglaAhorro: sin presencia -> ejecutarAccion("OFF") Justificación: Incorporado a la jerarquía y al sistema de reglas sin modificar ActuadorBombilla, ActuadorVentilador, ni la interfaz Regla.

SECCIÓN 6 — Estructura del Proyecto

El código fuente se organiza según la separación MVC y se gestiona con Git. La carpeta docs/ contiene la documentación Javadoc generada automáticamente.

```
SmartTecnoHouse-POO/
|- src/main/java/
|  |- modelo/          <- IDispositivo, Sensor, Actuador, clases concretas, Reglas, Singleton
|  |- vista/           <- VistaPrincipal, PanelSensores, PanelActuadores, PanelReglas
|  |- controlador/     <- Controlador.java
|  \- Main.java
|- data/
|  |- estado.json      <- Persistencia del estado de actuadores
|  \- actuators.log    <- Registro de acciones del sistema
|- docs/              <- Javadoc HTML generado automaticamente
|- README.md
\-.gitignore          <- Plantilla Java (excluye .class, /out/, /.idea/)
```

SECCIÓN 7 — Control de Versiones

Hash	Mensaje	Fecha
786bda6	Merge pull request #2 from hitosdesi/develop	Abr 2026
feat...	fix: eliminado ActuadorPersiana incompatible	Abr 2026
...	feat: prueba del sistema domótico con Main	Mar 2026
...	feat: estructura inicial de carpetas MVC	Mar 2026
...	Initial commit	Mar 2026

Repositorio: <https://github.com/hitosdesi/SmartTecnoHouse-POO>